UNIT 3.Association Rule Mining

Apriori Algorithm

What is Association Rule Mining?

Association Rule Mining is a **data mining technique** used to find relationships between items in large datasets.

- "If a customer buys Bread, what else do they buy?"

This is commonly used in **Market Basket Analysis**.

What is Apriori Algorithm?

Apriori Algorithm is the most popular algorithm used to find **frequent item sets** and generate **association rules**.

It works based on the principle:

Key Idea

If an itemset is frequent, all of its subsets must also be frequent.

This is called the **Apriori Property**.

Measures Used

Support

How frequently an item appears in the dataset.

$$Support(A) = \frac{Number\ of\ transactions\ containing\ A}{Total\ transactions}$$

Confidence

The probability that B happens when A has already happened**.**

$$Confidence(A \rightarrow B) = \frac{Support(A \cup B)}{Support(A)}$$

Lift

- How strong the relationship between A and B.
- It checks whether items are really related or just occurring together by coincidence.

$$Lift = \frac{Confidence(A \rightarrow B)}{Support(B)}$$

Example:

Transactions

TID	Items Purchased
T1	Bread, Milk, Butter
T2	Bread, Milk, Cheese
T3	Milk, Butter, Eggs
T4	Bread, Milk, Butter, Cheese
T5	Bread, Milk, Butter

Assume:

- **Minimum Support = 60%**
- Total transactions = 5
- Minimum support count = 3

Step 1: Frequent 1-Itemsets (L1)

Count support:

Item	Count
Bread	4
Milk	5
Butter	4

Item	Count
Cheese	2
Eggs	1
Juice	0

Remove items with support < 3.

Frequent 1-itemsets:

{Bread}, {Milk}, {Butter}

Step 2: Frequent 2-Itemsets (L2)

Combinations from L1:

1. {Bread, Milk} → 4
2. {Bread, Butter} → 3
3. {Milk, Butter} → 4

All satisfy minimum support.

Frequent 2-itemsets:

{Bread, Milk}

{Bread, Butter}

{Milk, Butter}

Step 3: Frequent 3-Itemsets (L3)

Combine L2:

{Bread, Milk, Butter}

Support Count (In transactions T1, T4, T5) → 3

it is also frequent.

No larger itemsets possible.

Step 4: Generate Association Rules

Rules generated from frequent itemsets.

From {Bread, Milk}

1. Bread → Milk

   Confidence = 4/4 = 100%

2. Milk → Bread
   Confidence = 4/5 = 80%

From {Milk, Butter}

1. Milk → Butter
   Confidence = 4/5 = 80%
2. Butter → Milk
   Confidence = 4/4 = 100%

From {Bread, Butter}

1. Bread → Butter
   Confidence = 3/4 = 75%
2. Butter → Bread
   Confidence = 3/4 = 75%

From {Bread, Milk, Butter}

Total possible rules = $2^3 - 2 = 6$

1. Bread, Milk → Butter
   Confidence = 3/4 = 75%
2. Bread, Butter → Milk
   Confidence = 3/3 = 100%
3. Milk, Butter → Bread
   Confidence = 3/4 = 75%
4. Bread → Milk, Butter
   Confidence = 3/4 = 75%
5. Milk → Bread, Butter
   Confidence = 3/5 = 60%
6. Butter → Bread, Milk
   Confidence = 3/4 = 75%

FP-Tree

The **FP-Tree Algorithm (Frequent Pattern Tree Algorithm)** is an efficient method used in **data mining** (especially in Data Mining) to find **frequent itemsets** without generating candidate sets like the Apriori algorithm.

What is FP-Tree?

An **FP-Tree (Frequent Pattern Tree)** is a compact data structure that stores important information about frequent patterns in a dataset.

Why FP-Tree?

- Avoids repeated database scans
- No candidate generation
- Faster than Apriori for large datasets

Transaction ID	Items
T1	{E,K,M,N,O,Y}
T2	{D,E,K,N,O,Y}
T3	{A,E,K,M}
T4	{C,K,M,U,Y}
T5	{C,E,I,K,O,O}

- The above-given data is a hypothetical dataset of transactions with each letter representing an item.
- Let the minimum support be 3.

Step-1

Item	Frequency
A	1
C	2
D	1
E	4
I	1
K	5
M	3
N	2
O	3
U	1
Y	3

A **Frequent Pattern set (L)** is built which will contain all the elements whose frequency is greater than or equal to the minimum support.

As minimum support be 3.

These elements are stored in descending order of their respective frequencies.

After insertion of the relevant items, the set L looks like this:- **L = {K : 5, E : 4, M : 3, O : 3, Y : 3}**

Step-2

Now, for each transaction, the respective **Ordered-Item set** is built.

Frequent Pattern set L = {K : 5, E : 4, M : 3, O : 3, Y : 3}

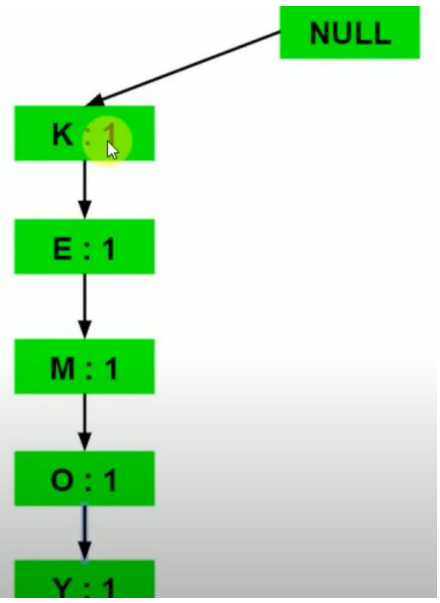
Transaction ID	Items	Ordered Item Set
T1	{E,K,M,N,O,Y}	{K,E,M,O,Y}
T2	{D,E,K,N,O,Y}	{K,E,O,Y}
T3	{A,E,K,M}	{K,E,M}
T4	{C,K,M,U,Y}	{K,M,Y}
T5	{C,E,I,K,O,O}	{K,E,O}

Step-3

Now, all the Ordered-Item sets are inserted into a Trie Data Structure.

Transaction ID	Items	Ordered-Item Set
T1	{E,K,M,N,O,Y}	{K,E,M,O,Y}
T2	{D,E,K,N,O,Y}	{K,E,O,Y}
T3	{A,E,K,M}	{K,E,M}
T4	{C,K,M,U,Y}	{K,M,Y}
T5	{C,E,I,K,O,O}	{K,E,O}

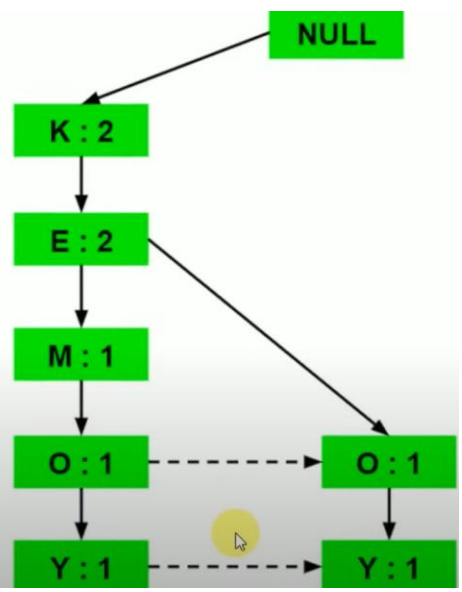
a) Inserting the set {K, E, M, O, Y}:



Now, all the Ordered-Item sets are inserted into a Trie Data Structure.

Transaction ID	Items	Ordered-Item Set
T1	{E,K,M,N,O,Y}	{K,E,M,O,Y}
T2	{D,E,K,N,O,Y}	{K,E,O,Y}
T3	{A,E,K,M}	{K,E,M}
T4	{C,K,M,U,Y}	{K,M,Y}
T5	{C,E,I,K,O,O}	{K,E,O}

b) Inserting the set {K, E, O, Y}:

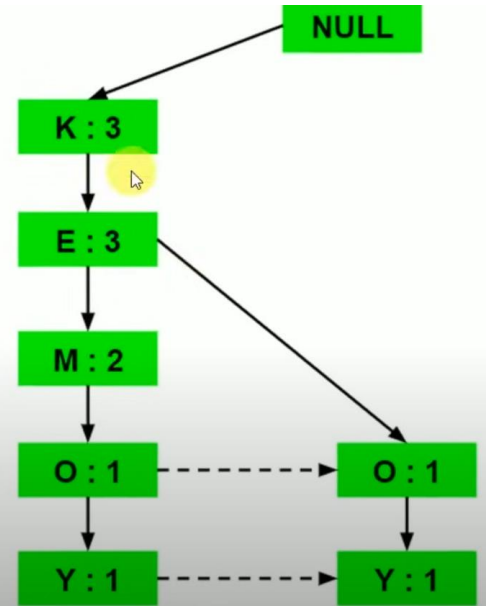


MORE VIDEOS

Now, all the Ordered-Item sets are inserted into a Trie Data Structure.

Transaction ID	Items	Ordered-Item Set
T1	{E,K,M,N,O,Y}	{K,E,M,O,Y}
T2	{D,E,K,N,O,Y}	{K,E,O,Y}
T3	{A,E,K,M}	{K,E,M}
T4	{C,K,M,U,Y}	{K,M,Y}
T5	{C,E,I,K,O,O}	{K,E,O}

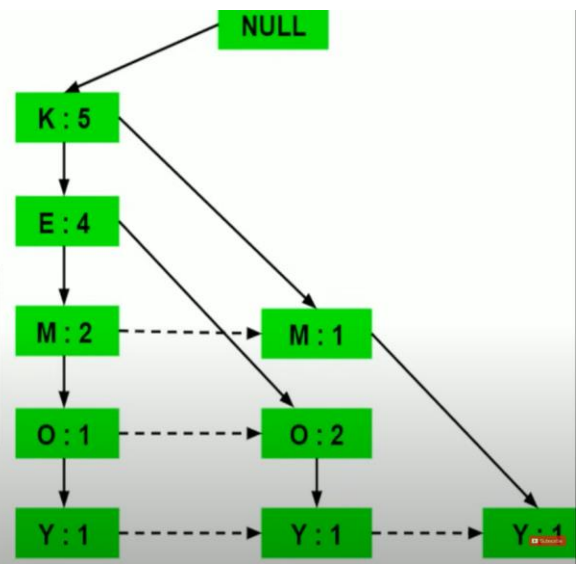
c) Inserting the set {K, E, M}:



Step-4

Now, for each item, the **Conditional Pattern Base** is computed which is path labels of all the paths which lead to any node of the given item in the frequent-pattern tree.

Items	Conditional Pattern Base
Y	{{K,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}
O	{{K,E,M : 1}, {K,E : 2}}
M	{{K,E : 2}, {K : 1}}
E	{K : 4}
K	



Step-5

Now for each item the **Conditional Frequent Pattern Tree is built.** It is done by taking the set of elements which is common in all the paths in the Conditional Pattern Base of that item and calculating it's support count by summing the support counts of all the paths in the Conditional Pattern Base.

Items	Conditional Pattern Base	Conditional Frequent Pattern Tree
Y	{{K,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}	{K : 3}
O	{{K,E,M : 1}, {K,E : 2}}	{K,E : 3}
M	{{K,E : 2}, {K : 1}}	{K : 3}
E	{K : 4}	{K : 4}
K		

Step-6

- From the Conditional Frequent Pattern tree, the **Frequent Pattern rules** are generated by pairing the items of the Conditional Frequent Pattern Tree set to the corresponding to the item as given in the below table.

Items	Conditional Pattern Base	Conditional Frequent Pattern Tree
Y	{{K,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}	{K : 3}
O	{{K,E,M : 1}, {K,E : 2}}	{K,E : 3}
M	{{K,E : 2}, {K : 1}}	{K : 3}
E	{K : 4}	{K : 4}
K		

Items	Frequent Pattern Generated
Y	{<K,Y : 3>}
O	{<K,O : 3>, <E,O : 3>, <E,K,O : 3>}
M	{<K,M : 3>}
E	{<E,K : 3>}
K	

MORE VIDEOS

Step-7

From Step – 6 output, create all possible association rules.

Example:

{K,Y:3} From this the possible rules are

K -> Y

Y -> K

Multidimensional Association Rules

1. Definition

Multidimensional association rules are rules that involve more than one attribute from a relational database.

Instead of using only one field, these rules combine multiple attributes such as **Age, Income, Location, Product, etc.**

Example:

(Age=Young, Income=High) → Product=Laptop

2. Key Rules

Each attribute can take only one value in a transaction

Example:

- Invalid combination:
  (Age = Young, Age = Middle)
- Valid combination:
  (Age = Young, Income = High)

Use of Apriori Property

- Based on the **Apriori principle**:

  "All subsets of a frequent itemset must also be frequent."

3. Problem

Given Transaction Table

TID	Age	Income	Product
1	Young	High	Laptop
2	Young	Medium	Mobile
3	Middle	High	Laptop
4	Young	High	Laptop
5	Middle	Medium	Tablet

Minimum Support = 40%

Support Count = 2

Total Transactions = 5

4. Item Representation

Represent the items in the table as given below for easy access.

Item	Meaning
i1	Age = Young
i2	Age = Middle
i3	Income = High
i4	Income = Medium
i5	Product = Laptop
i6	Product = Mobile
i7	Product = Tablet

5. Solution

Step 1: C1 (1-itemsets)

Support Count

Item	Count
i1	3
i2	2
i3	3
i4	2
i5	3
i6	1
i7	1

Consider only the items which has the support count as 2 or more.

L1 (Frequent Item sets)

✓ Keep: i1, i2, i3, i4, i5

✕ Remove: i6, i7

Step 2: C2 (2-itemsets)

Valid Combinations (different attributes only)

(i1, i3)

(i1, i4)

(i2, i3)

(i2, i4)

(i1, i5)

(i3, i5)

Note:

(i1, i2) not allowed (same attribute)

Support Count

Item set	Count
(i1, i3)	2
(i1, i4)	1
(i2, i3)	1
(i2, i4)	1
(i1, i5)	2
(i3, i5)	3

Consider only the items which has the support count as 2 or more.

L2 (Frequent)

✓ (i1, i3), (i1, i5), (i3, i5)

✕ Others removed

Step 3: C3 (3-itemsets)

Possible Combination

(i1, i3, i5)

Support Count

Itemset	Count
(i1, i3, i5)	2

Consider only the items which has the support count as 2 or more.

L3(Frequent)

✓ (i1, i3, i5)

6. Rule Generation

Final Rule

(i1, i3) → i5

Calculations

Confidence = 2/2 = 100%

7. Final Answer

Frequent Item sets

L1 = {i1, i2, i3, i4, i5}

L2 = {(i1,i3), (i1,i5), (i3,i5)}

L3 = {(i1,i3,i5)}

Strong Rule

(i1, i3) → i5

Confidence = 100%